

PROJECT SYNOPSIS

Student Details

Institute/University	Pimpri Chinchwad College of Engineering (PCCOE), Pune
Programme / Batch	B.Tech Computer Science and Engineering (AI & ML) / 2027 Batch
Student Name	Rupam Gunavant Karale
PRN	123B1B164

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5)	CS4
Project Title / Topic	Secure Code Debugging and Review Assistant

2. Objectives

The project focuses on reducing the manual effort of reviewing embedded C code for security and reliability defects, while keeping detection deterministic and every decision traceable to a person.

- Build an assistant that helps engineers find, understand and review risky patterns in C code, such as unbounded copies, unchecked allocations and hard-coded credentials.
- Use RAG to answer questions and explain findings from the supplied code and an approved set of coding rules, with citations to file and line range and to rule ID.
- Use a local LLM to explain root causes, suggest fixes and analyse compiler or static-analysis log lines.
- Keep detection deterministic through Python rules, and record an engineer's accept or reject decision for every finding with CSV and JSON export.

3. Scope

The scope is intentionally limited to a controlled prototype using synthetic content, deterministic detection and human review.

In Scope – To Be Implemented

- A synthetic C source file with seeded defects and a set of 8 synthetic coding rules used as the knowledge base.
- A deterministic scanner with 8 rules, RAG-based code Q&A, AI explanation of findings, and log / warning analysis.
- Local/private processing using Ollama, Sentence Transformers and FAISS, with a Streamlit interface and Python backend modules.
- Human review (accept / reject), a SQLite review log and CSV/JSON export.

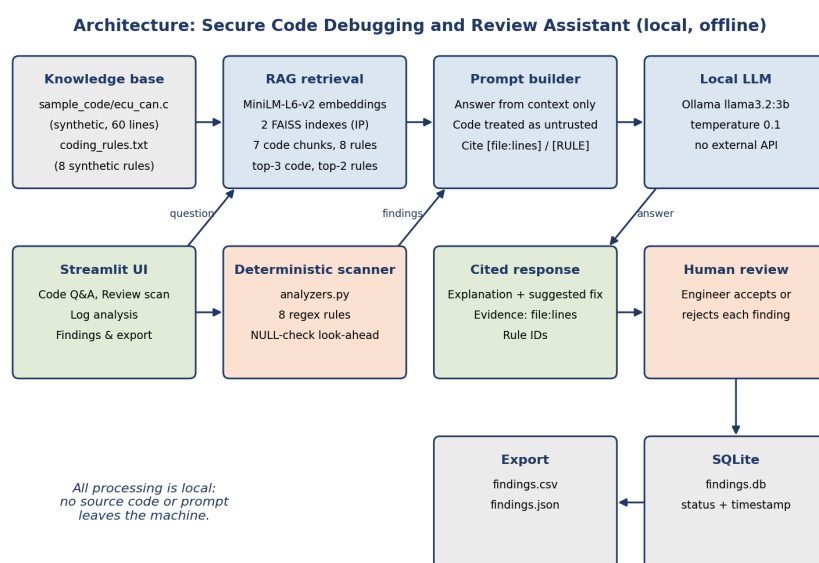
Out of Scope – Will Not Be Implemented

- Automatic fixing or merging of code, and execution of any code under review.
- Multi-file project analysis, full MISRA or CERT compliance claims, and use of proprietary standards or OEM source code.
- Production-scale deployment in the current prototype.

4. Proposed Solution Design

Component	Planned choice
Pre-trained model (name, version, local/private hosting)	Ollama running locally with llama3.2:3b. Used for grounded answers, finding explanations and log analysis. Local execution keeps the prototype independent of an external LLM API.
Embedding model	Sentence Transformers all-MiniLM-L6-v2, used to embed code chunks, rules and user questions for semantic retrieval.
Vector store (FAISS / ChromaDB)	FAISS with two local indexes (code chunks and coding rules).
RAG pipeline and/or AI-agent design – brief description	The C file is split into 12-line windows with 4-line overlap and each rule is one chunk. For a query, the top 3 code chunks and top 2 rules are retrieved and passed to the local LLM. The prompt instructs the model to answer only from the supplied context, to treat the code as untrusted, and to cite [FILE:lines] and [RULE:ID]. No autonomous agent is used.
Service layer and UI (e.g., FastAPI / Flask / Streamlit)	Streamlit interface with four tabs (Code Q&A, Review scan, Log / warning analysis, Findings & export) and Python modules for scanning, retrieval and storage. SQLite stores the review log.
Tools / deterministic rules, if applicable	Eight regular-expression rules (for example gets(), strcpy, hard-coded credentials, format strings, unchecked malloc and fopen). Detection is rule-based; the LLM only explains findings.

Proposed System / Architecture



System architecture (also included in the technical report).

5. Input Data / Knowledge Base

Only synthetic project content is used as the knowledge base.

Data / Knowledge Sources	A synthetic C file modelled on a CAN-frame ECU module (ecu_can.c) with 8 deliberately seeded defects and one correct function used as a negative control, and a synthetic coding-rules file (coding_rules.txt) with 8 rules. No live or proprietary data is required.
Approximate Size and Format	ecu_can.c: about 60 lines of C. coding_rules.txt: 8 text rules with ID, severity and instruction. Findings are stored in local SQLite and can be exported as CSV and JSON.

Preprocessing and EDA Planned	Text is read and split into line-window chunks with file and line metadata; each rule becomes one chunk. Embeddings are created with all-MiniLM-L6-v2 and indexed with FAISS. Numerical EDA is not required because the knowledge source is source code and rule text.
--------------------------------------	--

6. Expected Outcomes and Evaluation Plan

Expected Outputs – aligned to the case study	A list of detected defects with file, line, rule and severity; cited AI explanations with suggested fixes; analysis of pasted log lines; and a stored, exportable record of every engineer decision (accepted or rejected). The system is intended to reduce manual review effort while detection and final decisions stay deterministic and human-controlled.
Evaluation Method – test question set / ground truth / metrics / validation approach	Ground truth is the set of 8 seeded defects. Scanner precision and recall are measured, including false positives on the negative-control function. Retrieval is measured as the hit rate of the expected function in the top-3 chunks for a set of test questions. AI explanations are checked manually for correctness and citation use. An evaluation script (run_eval.py) will produce the results.
Responsible AI Measures – grounding and citations / privacy / security / human oversight	Grounding: answers use retrieved code and rules and cite them. Privacy: the LLM, embeddings and vector store run locally. Security: code under review is treated as untrusted in the prompt; nothing is executed or changed automatically. Human oversight: every finding is accepted or rejected by an engineer and logged. Only synthetic data is used.
MLOps / Deployment Elements Planned – versioning / Docker / tracking	Git/GitHub for source versioning, a Python virtual environment for dependency isolation, and requirements.txt to record dependencies. The prototype runs locally; Docker, CI/CD and formal experiment tracking are future improvements.

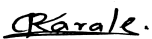
7. Planned AI-Tool Usage

AI tools are used as development assistance and as a local model inside the application; the student reviews the resulting implementation.

AI Tool / Model	Purpose	Artifact Affected
Claude (Anthropic)	Coding assistance, generation of the synthetic C file and rules, debugging, and documentation drafting.	Python modules, synthetic knowledge base, synopsis and technical report. All assisted content is reviewed and verified by the student.
Ollama / llama3.2:3b	Local project LLM for Q&A, finding explanations and log analysis.	RAG/AI modules and generated explanation content.

Student Declaration

- ☒ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.
- ☒ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Signature:		Date:	07-10-2026
Name:	Rupam Gunavant Karale	PRN:	123B1B164

Faculty Approval Record

Project title	Secure Code Debugging and Review Assistant (CS4)
Student	Rupam Gunavant Karale, PRN 123B1B164
Faculty / mentor name	
Decision	☐ Approved ☐ Approved with changes ☐ Not approved
Remarks	
Faculty signature and date	